

Quy hoạch động nâng cao trên dãy & xâu

Nội dung

Trong khóa Đồng, bạn đã học DP cơ bản: Fibonacci, coin change, grid paths, knapsack 0/1, LIS bản dễ. Bài này nâng cấp những kỹ năng đó với:

1. LIS $O(n \log n)$ — patience sorting
2. LCS — Longest Common Subsequence
 - Biến thể quen thuộc: Edit Distance (Levenshtein)
3. Knapsack mở rộng
 - 3.1. Unbounded Knapsack — vật được lấy nhiều lần
 - 3.2. Bounded Knapsack — vật i có c_i bản
 - 3.3. Group Knapsack — mỗi nhóm chọn ≤ 1 vật
4. DP đếm trên dãy / xâu
 - Ví dụ: Xâu palindrome con dài nhất (LPS)
 - Ví dụ: Tìm xâu thứ k theo thứ tự từ điển
5. Mẹo DP nâng cao
6. Bẫy thường gặp
7. Tổng kết
 - LIS — dùng binary search trên patience array $O(n \log n)$
 - LCS / Edit Distance — DP trên hai xâu
 - Knapsack mở rộng: unbounded, bounded, group
 - DP đếm trên dãy / xâu — palindrome, đếm cách

1. LIS $O(n \log n)$ — patience sorting

Nhắc lại bài toán: Cho dãy $a_1, \dots, a_n$. Tìm dãy con tăng dài nhất.

- Cách $O(n^2)$ (Bronze): $dp[i] = \text{LIS kết thúc ở } i$.
- Cách $O(n \log n)$: Duy trì mảng `tail`, trong đó `tail[k]` = giá trị nhỏ nhất có thể là phần tử cuối của một LIS độ dài $k + 1$.

Tính chất: `tail` luôn tăng nghiêm ngặt.

Quy trình: Với mỗi a_i :

- Nếu $a_i >$ mọi giá trị trong `tail` $\rightarrow$ append.
- Ngược lại, tìm vị trí đầu tiên $\geq a_i$ trong `tail` và thay giá trị đó bằng a_i .

C++

```
vector<int> tail;
for (int x : a) {
    auto it = lower_bound(tail.begin(), tail.end(), x);
    if (it == tail.end()) tail.push_back(x);
    else *it = x;
}
cout << tail.size();
```

Python

```
import bisect
tail = []
for x in a:
    i = bisect.bisect_left(tail, x)
    if i == len(tail):
        tail.append(x)
    else:
        tail[i] = x
print(len(tail))
```

Đáp án = `tail.size()`.

Lưu ý:

- `lower_bound` cho LIS nghiêm ngặt (strictly increasing).
- Đổi sang `upper_bound` cho không giảm (non-decreasing).

Cảnh báo: `tail` không phải chính LIS — chỉ độ dài đúng. Để khôi phục LIS, cần lưu thêm thông tin (parent pointers).

2. LCS — Longest Common Subsequence

Đề: Cho hai chuỗi A độ dài n , B độ dài m . Tìm chuỗi con dài nhất xuất hiện trong cả hai.

Trạng thái: $dp[i][j]$ = LCS của $A[1..i]$ và $B[1..j]$.

Truy hồi:

- Nếu $A[i] = B[j]$: $dp[i][j] = dp[i-1][j-1] + 1$.
- Ngược lại: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

C++

```
int dp[N][N];
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        if (a[i] == b[j]) dp[i][j] = dp[i-1][j-1] + 1;
        else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
    }
cout << dp[n][m];
```

Python

```
dp = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i] == b[j]:
            dp[i][j] = dp[i-1][j-1] + 1
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
print(dp[n][m])
```

Phục hồi LCS: truy ngược từ $dp[n][m]$ — nếu ký tự bằng nhau và $dp[i-1][j-1] + 1 = dp[i][j]$ thì lấy ký tự đó, lùi cả hai. Ngược lại, lùi theo hướng có dp lớn hơn.

Phức tạp: $O(nm)$, nhớ $O(nm)$ — có thể giảm xuống $O(\min(n, m))$ nếu chỉ cần độ dài.

Biến thể quen thuộc: Edit Distance (Levenshtein)

Cùng khung 2D trên hai xâu, chỉ thay truy hồi. Edit distance đếm số phép biến đổi (chèn / xóa / sửa) tối thiểu để biến $A \rightarrow B$:

- $dp[0][j] = j$, $dp[i][0] = i$.
- Nếu $A[i] = B[j]$: $dp[i][j] = dp[i-1][j-1]$.
- Ngược lại: $dp[i][j] = 1 + \min(\{dp[i-1][j], dp[i][j-1], dp[i-1][j-1]\})$ (xóa / chèn / sửa).

C++

```
int dp[N][N];
for (int i = 0; i <= n; i++) dp[i][0] = i;
for (int j = 0; j <= m; j++) dp[0][j] = j;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        if (a[i] == b[j]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    }
cout << dp[n][m];
```

Python

```
dp = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(n + 1): dp[i][0] = i
for j in range(m + 1): dp[0][j] = j
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i] == b[j]:
            dp[i][j] = dp[i-1][j-1]
        else:
            dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
print(dp[n][m])
```

Điểm mấu chốt: mọi DP 2D trên hai xâu đều cùng khung này, chỉ khác công thức truy hồi.

3. Knapsack mở rộng

3.1. Unbounded Knapsack — vật được lấy nhiều lần

Khác 0/1 knapsack: đảo chiều vòng lặp j thành tăng.

C++

```
// 0/1 - duyệt j từ W về w[i] (giảm)
for (int i = 1; i <= n; i++)
    for (int j = W; j >= w[i]; j--)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);

// Unbounded - duyệt j từ w[i] tới W (tăng)
for (int i = 1; i <= n; i++)
    for (int j = w[i]; j <= W; j++)
        dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
```

Python

```
# 0/1
dp = [0] * (W + 1)
for i in range(1, n + 1):
    for j in range(W, w[i] - 1, -1): # giảm dần
        dp[j] = max(dp[j], dp[j - w[i]] + v[i])

# Unbounded
dp = [0] * (W + 1)
for i in range(1, n + 1):
    for j in range(w[i], W + 1): # tăng dần
        dp[j] = max(dp[j], dp[j - w[i]] + v[i])
```

Lý do: với unbounded, khi cập nhật $dp[j]$ ta dùng $dp[j - w[i]]$ ở lượt hiện tại (đã cập nhật) — tức cho phép chọn vật i nhiều lần.

3.2. Bounded Knapsack — vật i có c_i bản

Cách ngây thơ: nhân ra thành $\sum c_i$ vật riêng $\rightarrow O(nW \sum c_i)$ — chậm.

Tối ưu binary: phân tách c_i thành các "gói" có kích thước $1, 2, 4, 8, \dots$ và một gói "dư" để tổng vừa đúng c_i . Mọi cách chọn $0..c_i$ vật đều có thể biểu diễn là tổng các gói được chọn (giống biểu diễn nhị phân). Tổng số gói trên tất cả vật là $O(n \log C) \rightarrow$ giải bằng 0/1 knapsack thường.

C++

```
// items[k] = (weight_k, value_k) sau khi phân tách
vector<pair<int,int>> items;
for (int i = 1; i <= n; i++) {
    int cnt = c[i];
    for (int k = 1; k <= cnt; k *= 2) {
        int take = min(k, cnt);
        items.push_back({take * w[i], take * v[i]});
        cnt -= take;
    }
    if (cnt > 0) items.push_back({cnt * w[i], cnt * v[i]});
}
// Sau đó chạy 0/1 knapsack thường trên items
vector<int> dp(W + 1, 0);
for (auto [wi, vi] : items)
    for (int j = W; j >= wi; j--)
        dp[j] = max(dp[j], dp[j - wi] + vi);
cout << dp[W];
```

Python

```
items = []
for i in range(1, n + 1):
    cnt = c[i]
    k = 1
    while k <= cnt:
        take = min(k, cnt)
        items.append((take * w[i], take * v[i]))
        cnt -= take
        k *= 2
    if cnt > 0:
        items.append((cnt * w[i], cnt * v[i]))

dp = [0] * (W + 1)
for wi, vi in items:
    for j in range(W, wi - 1, -1):
```

```

        dp[j] = max(dp[j], dp[j - wi] + vi)
    print(dp[W])

```

3.3. Group Knapsack — mỗi nhóm chọn ≤ 1 vật

C++

```

for (auto& group : groups) {
    for (int j = W; j >= 0; j--)
        for (auto [w, v] : group)
            if (j >= w) dp[j] = max(dp[j], dp[j-w] + v);
}

```

Python

```

for group in groups:
    for j in range(W, -1, -1):
        for w, v in group:
            if j >= w:
                dp[j] = max(dp[j], dp[j-w] + v)

```

Lưu ý: vòng j ở ngoài vòng "vật trong nhóm" để tránh chọn 2 vật cùng nhóm.

4. DP đếm trên dãy / chuỗi

Nhiều bài DP không hỏi giá trị tối ưu mà hỏi số cách. Khung tư duy như cũ — chỉ thay max/min bằng +.

Ví dụ: Xâu palindrome con dài nhất (LPS)

Cho xâu s độ dài n . Tìm xâu con (subsequence) là palindrome dài nhất.

Trạng thái: $dp[i][j]$ = LPS của $s[i..j]$.

Truy hồi:

- Nếu $i = j$: $dp[i][j] = 1$.
- Nếu $s[i] = s[j]$: $dp[i][j] = dp[i+1][j-1] + 2$.
- Ngược lại: $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$.

C++

```

// s[1..n] - chỉ số từ 1 (giống LCS ở §2)
for (int len = 1; len <= n; len++)
    for (int i = 1; i + len - 1 <= n; i++) {
        int j = i + len - 1;
        if (i == j) dp[i][j] = 1;
        else if (s[i] == s[j]) dp[i][j] = dp[i+1][j-1] + 2;
    }

```

```

        else dp[i][j] = max(dp[i+1][j], dp[i][j-1]);
    }
    cout << dp[1][n];

```

Python

```

# s[1..n] - chỉ số từ 1
dp = [[0] * (n + 2) for _ in range(n + 2)]
for length in range(1, n + 1):
    for i in range(1, n - length + 2):
        j = i + length - 1
        if i == j:
            dp[i][j] = 1
        elif s[i] == s[j]:
            dp[i][j] = dp[i+1][j-1] + 2
        else:
            dp[i][j] = max(dp[i+1][j], dp[i][j-1])
print(dp[1][n])

```

Quan trọng: thứ tự duyệt theo độ dài đoạn (`len`) — vì $dp[i][j]$ phụ thuộc đoạn ngắn hơn.

Ví dụ: Tìm xâu thứ k theo thứ tự từ điển

Đây là một kỹ thuật rất kinh điển kết hợp DP đếm và xây dựng đáp án từng bước.

Đề mẫu: Cho ba số m, n, p . Tập S gồm tất cả các xâu chỉ chứa đúng m ký tự X, n ký tự Y, p ký tự Z. Tìm xâu thứ k trong S theo thứ tự từ điển.

Bước 1 — Đếm bằng DP: Gọi $f(m, n, p)$ = số xâu có thể tạo từ m ký tự X, n ký tự Y, p ký tự Z. Truy hồi:

$$f(m, n, p) = f(m - 1, n, p) + f(m, n - 1, p) + f(m, n, p - 1)$$

(điều kiện biên $f(0, 0, 0) = 1$). Đây chính là số tổ hợp $\frac{(m+n+p)!}{m!n!p!}$.

Bước 2 — Xây dựng xâu từng ký tự: Tại mỗi vị trí, thử ký tự nhỏ nhất trước ($X < Y < Z$). Đếm xem có bao nhiêu xâu bắt đầu bằng ký tự đó:

- Nếu chọn X đầu tiên $\rightarrow$ còn $m - 1$ ký tự X, n ký tự Y, p ký tự Z $\rightarrow$ có $f(m - 1, n, p)$ xâu.
- Nếu $k \leq f(m - 1, n, p) \rightarrow$ đáp án bắt đầu bằng X, đệ quy với $(m - 1, n, p)$ và giữ nguyên k .
- Ngược lại $\rightarrow$ đáp án không bắt đầu bằng X. Trừ k đi $f(m - 1, n, p)$ và thử ký tự kế tiếp Y.

C++

```

string solve(int m, int n, int p, long long k) {
    string ans;
    while (m + n + p > 0) {
        long long cntX = (m > 0) ? f[m-1][n][p] : 0;
        long long cntY = (n > 0) ? f[m][n-1][p] : 0;
        if (k <= cntX) { ans += 'X'; m--; }
        else if (k <= cntX + cntY) { k -= cntX; ans += 'Y'; n--; }
        else { k -= cntX + cntY; ans += 'Z'; p--; }
    }
    return ans;
}

```

Python

```

def solve(m, n, p, k):
    ans = []
    while m + n + p > 0:
        cntX = f[m-1][n][p] if m > 0 else 0
        cntY = f[m][n-1][p] if n > 0 else 0
        if k <= cntX:
            ans.append('X'); m -= 1
        elif k <= cntX + cntY:
            k -= cntX; ans.append('Y'); n -= 1
        else:
            k -= cntX + cntY; ans.append('Z'); p -= 1
    return ''.join(ans)

```

Tại sao đúng? Tất cả các xâu trong S được sắp theo thứ tự từ điển. Khi đếm "có bao nhiêu xâu bắt đầu bằng X ", ta biết chính xác các vị trí thứ tự 1 đến $f(m-1, n, p)$ trong S — mọi xâu thứ tự đó đều bắt đầu bằng X . Còn các xâu sau đó thì không. Việc trừ k chính là đổi hệ tham chiếu sang nhóm còn lại.

Bẫy quan trọng:

- k có thể rất lớn — thường cần `long long` hoặc `BigInt`.
- Đôi khi đề yêu cầu chặn đếm ở 10^{18} để tránh tràn (nếu k tối đa 10^{18} , ta không cần biết chính xác f khi nó vượt 10^{18}).

Khi nào áp dụng được?

- Có thể đếm số phần tử của tập "có tiền tố T " trong thời gian hợp lý.
- Tập có thứ tự rõ ràng (từ điển, độ dài tăng dần, v.v.).

Đây là kỹ thuật rất đáng nhớ — bạn sẽ gặp nó trong nhiều bài "tìm đáp án thứ k ".

5. Mẹo DP nâng cao

- **Tối ưu bộ nhớ:** nhiều DP chỉ phụ thuộc 2 hàng $\rightarrow$ giảm $O(nm)$ xuống $O(m)$.
- **Tự hỏi:** trạng thái là gì? Truy hồi gì? Thứ tự duyệt nào đúng?
- **Vẽ bảng DP:** cực kỳ hữu ích để debug.
- **Khôi phục đáp án:** lưu thêm "lựa chọn" (chọn vật nào, bước nào) để truy ngược.

6. Bẫy thường gặp

- 0/1 knapsack vs unbounded: chiều duyệt j ngược nhau!
- LCS index: dễ nhầm i với $i - 1$.
- Edit distance khởi tạo: phải đặt $dp[i][0] = i$ và $dp[0][j] = j$.
- Tràn long long: tích lớn $\rightarrow$ dùng long long.

7. Tổng kết

BÀI TOÁN	TRẠNG THÁI	PHỨC TẠP
LIS chuẩn	$dp[i]$	$O(n^2)$
LIS nhanh	patience array	$O(n \log n)$
LCS	$dp[i][j]$	$O(nm)$
Edit distance	$dp[i][j]$	$O(nm)$
0/1 knapsack	$dp[j]$	$O(nW)$
Unbounded knapsack	$dp[j]$	$O(nW)$
LPS / palindrome con	$dp[i][j]$	$O(n^2)$